

Project Report

On

(DeepSea-YOLO: Automated Underwater Pollution Detection Framework)

By

Name of the Student (DHAKSHESH B)

(Register No:RA2432242020024)

Submitted to the

PG DEPARTMENT OF COMPUTER APPLICATIONS

Under the guidance of

(Dr. S. MEENAKSHI M.Sc.,M.Phil.,Ph.D.,)

(ASSISTANT PROFESSOR, PG Department of Computer Applications)

Submitted in partial fulfillment of the

requirement for the award of the degree of

**MASTER OF COMPUTER APPLICATIONS –
GENERATIVE AI**



**SRM Institute of Science and Technology
Chennai Ramapuram
APRIL 2026**



SRM Institute of Science and Technology
Chennai Ramapuram
Faculty of Science & Humanities
(A Place for Transformation)
PG Department of Computer Applications



BONAFIDE CERTIFICATE

Certified that the project report titled “**DeepSea-YOLO: Automated Underwater Pollution Detection Framework**“ is the bonafide work of **Mr DHAKSHESH B (Register No:RA2432242020024)** who carried out the Project work (PGI20P03L) done under my supervision.

Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Signature of the Internal Guide

Signature of Head of the Department

Signature of the External Examiner

(Plagiarism report of the Project Report, **it should be below 10%**)

DATE: 10.04.26

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Dhakshesh B (Reg. No.RA2432242020024)**, student of **SRM Institute Of Science And Technology, Ramapuram. MCA Generative AI**, has completed the project entitled **“Real-Time Detection And Analysis Of Underwater Objects Using An Optimized YOLO Model”** in our concern during the period of **December 2025 to April 2026**.

We wish them all the best in all their future endeavors.



M. Arjun
Authorized Signature

DeepSea-YOLO: Automated Underwater Pollution Detection Framework

ABSTRACT

Marine pollution, particularly from plastic waste and synthetic debris, has emerged as a critical global environmental issue that severely impacts aquatic ecosystems, marine biodiversity, and human health. Plastics such as bottles, fishing nets, disposable face masks, and gloves accumulate in oceans and water bodies, causing long-term ecological damage and disrupting marine food chains. Traditional methods of monitoring marine waste rely heavily on manual inspection and cleanup operations, which are time-consuming, labor-intensive, and inefficient for large-scale environmental surveillance. To address these challenges, this project proposes a Smart Underwater Waste Detection System that leverages advanced deep learning techniques for automated marine pollutant identification.

The proposed system utilizes the YOLO (You Only Look Once) object detection algorithm, a state-of-the-art real-time deep learning model known for its speed and accuracy in detecting multiple objects within a single frame. The system is capable of identifying various types of underwater waste, including plastic bottles, fishing nets, face masks, and gloves, from images, recorded videos, and real-time camera feeds. By generating annotated outputs with bounding boxes and confidence scores, the model provides precise localization and classification of marine debris.

The application is developed using the Flask web framework, which ensures a lightweight and scalable backend architecture. A MySQL database is integrated as the backend storage system to maintain structured records of user data, uploaded media files, detection results, and session statistics. Secure user authentication is implemented through login and signup modules to ensure controlled access and data security. This structured database management approach enables efficient storage, retrieval, and analysis of detection data for long-term environmental monitoring. By integrating deep learning, web technologies, and database systems, this project demonstrates a practical and scalable solution for marine pollution monitoring. In the future, the system can be integrated with IoT devices, underwater drones, and autonomous marine robots to enable large-scale, real-time environmental surveillance and contribute toward sustainable ocean conservation efforts.

ACKNOWLEDGEMENT

I extend my sincere gratitude to the Chancellor, **Dr. T. R. PACHAMUTHU** and to Chairman **Dr. R. SHIVAKUMAR** of SRM Institute of Science and Technology, Ramapuram and Trichy campuses for providing me the opportunity to pursue the MCA-Generative AI degree at this University.

I express my sincere gratitude to **Dr. S. THIRUMAGAN, Dean(S&H), SRM IST, Ramapuram** for his support and encouragement for the successful completion of the project.

I record my heartfelt gratitude to **Dr. J. DHILIPAN, Vice Principal-Admin (S&H), SRM IST, Ramapuram**, for his continuous support and keen interest to make this project a successful one.

I extend my sincere thanks to **Dr. N. PUGHAZENDI, Vice Principal – Academics (S&H), SRM IST, Ramapuram**, for his unwavering support and invaluable guidance that greatly contributed to the success of this project.

I would like to convey my heartfelt gratitude to **Dr. R. AGUSTHIYAR, Professor and Head, PG Department of Computer Applications** for his useful advice and suggestions which really helpful to me during the project completion.

I find no word to express my profound gratitude to my guide <**Dr. S. MEENAKSHI M.Sc.,M.Phil.,Ph.D.,**>, < **ASSISTANT PROFESSOR** >, **PG Department of Computer Applications**, SRM IST Ramapuram.

I thank the almighty who has made this possible. Finally, I thank my beloved family members and friends for their motivation, encouragement and cooperation in all aspect which led me to the completion of this project.

DHAKSHESH B

(RA2432242020024)

LIST OF TABLES

S.NO	Table No.	Table Name	Page No.
1	1	User Table	26
2	2	Uploaded Files Table	27
3	3	Detection Result Table	27
4	4	Detection Summary Table	28

LIST OF FIGURES

S. No	Fig. No	Figures Name	Page No.
1	4.1	DataFlow Diagram	23
2	4.2	Architecture Diagram	25
3	8.1	Flask Home Page Interface	47

TABLE OF CONTENTS

S.NO	TITLE	PAGENO
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	LIST OF TABLES	iii
	LIST OF FIGURES	iv
CHAPTERS	TITLE	PAGENO.
1	INTRODUCTION 1.1PROJECT INTRODUCTION	2
2	WORKING ENVIRONMENT 2.1 HARDWARE REQUIREMENT 2.2 SOFTWARE REQUIREMENT 2.3 SYSTEM SOFTWARE	5
3	SYSTEM ANALYSIS 3.1 FEASIBILITY STUDY 3.2 EXISTING SYSTEM 3.3 DRAWBACKS OF EXISTING SYSTEM 3.4 PROPOSED SYSTEM 3.5 BENEFITS OF PROPOSED SYSTEM 3.6 SCOPE OF THE PROJECT	10

4	SYSTEM DESIGN 4.1 DATA FLOW DIAGRAM 4.2 ARCHITECTURE DIAGRAM 4.3 DATABASE DESIGN	23
5	PROJECT DESCRIPTION 5.1 OBJECTIVE 5.2 MODULE DESCRIPTION 5.3 IMPLEMENTATION	30
6	SYSTEM TESTING 6.1 TESTING DEFINITION 6.2 TESTING OBJECTIVE 6.3 TYPES OF TESTING	39
7	CONCLUSION 7.1 SUMMARY 7.2 FUTURE ENHANCEMENTS	43
8	APPENDIX 8.1 SCREEN SHOTS 8.2 CODING	47
9	BIBLIOGRAPHY AND REFERENCES	60

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 PROJECT INTRODUCTION :

Marine pollution has become one of the most pressing environmental challenges of the 21st century. Among the various forms of pollution, plastic waste and synthetic debris pose a serious threat to marine ecosystems, aquatic biodiversity, and human livelihoods. Every year, millions of tons of plastic materials such as bottles, packaging materials, fishing nets, disposable face masks, and gloves enter oceans and water bodies. These pollutants do not decompose easily and remain in the environment for decades, breaking down into microplastics that contaminate marine food chains. As a result, marine organisms suffer from ingestion, entanglement, habitat destruction, and toxic exposure.

Traditional marine waste monitoring methods rely largely on manual surveys, underwater divers, satellite observation, and physical cleanup operations. While these approaches contribute to environmental conservation, they are often labor-intensive, time-consuming, costly, and limited in scale. Monitoring underwater waste is particularly challenging due to poor visibility, varying light conditions, water turbidity, and large oceanic coverage. Therefore, there is a strong need for intelligent, automated, and scalable systems capable of detecting and analyzing marine waste efficiently.

Recent advancements in Artificial Intelligence (AI) and Computer Vision have opened new possibilities for environmental monitoring. Deep learning algorithms, especially object detection models, have shown remarkable performance in identifying and classifying objects in images and videos. Among these, the YOLO (You Only Look Once) algorithm stands out for its ability to perform real-time object detection with high accuracy and speed. YOLO processes an entire image in a single pass, making it highly efficient for applications requiring live monitoring and fast detection.

This project introduces a **Smart Underwater Waste Detection System** that utilizes the YOLO deep learning algorithm to automatically detect marine pollutants such as plastic bottles, fishing nets, face masks, and gloves from underwater images, videos, and real-time camera feeds. The system is designed to provide accurate object localization using bounding boxes along with confidence scores, ensuring reliable pollutant identification.

The application is developed using the Flask web framework, providing a lightweight and scalable backend architecture. A MySQL database is integrated to store user details, uploaded media files, detection results, and session statistics in a structured manner. Secure authentication mechanisms, including login and signup modules, ensure controlled access to the system and protect user data.

To address the challenges posed by underwater environments, preprocessing techniques such as contrast enhancement and noise reduction are applied to improve image clarity and detection accuracy. The system supports multiple operational modes, including image-based detection, video analysis with statistical reporting, and real-time detection through webcam input.

By combining deep learning, web development, and database management technologies, this project offers a practical and efficient solution for marine waste monitoring. The proposed system not only enhances environmental surveillance capabilities but also provides a foundation for future integration with IoT devices, underwater drones, and autonomous marine robots for large-scale ocean conservation efforts.

CHAPTER 2

WORKING ENVIRONMENT

CHAPTER 2

WORKING ENVIRONMENT

2.1 HARDWARE REQUIREMENT:

Operating System	:	Windows 7 or Higher
Processor	:	Minimum Dual-core 2.0 GHz ,Or Higher
Ram	:	Minimum 4 GB Or Higher
Hard Disc	:	Minimum 100 GB HDD or Higher
Display	:	Minimum: 1366×768 resolution Or Higher

2.2 SOFTWARE REQUIREMENT:

Programming Language	:	Python 3.8 Or Higher
Frameworks & Libraries	:	Flask , Pytorch , OpenCV
Editor Tools	:	VS code (Visual Studio Code)
Deployment Tools	:	Flask, PyCharm
Web Browser	:	Chrome / FireFox Browser

2.3 SYSTEM SOFTWARE:

The system software provides the fundamental platform required to develop, execute, and manage the Real-Time Detection and Analysis of Underwater Objects using an Optimized YOLO Model. It acts as the backbone of the entire application by managing hardware resources, enabling communication between software components, and supporting the execution of machine learning models. Since this project focuses on computer vision and real-time object detection, the system software must support deep learning frameworks, image processing libraries, and efficient runtime environments.

The project is primarily implemented using Python, which is widely used for machine learning and computer vision applications. Python offers a rich ecosystem of libraries and frameworks that simplify complex tasks such as image processing, model training, and real-time detection. Libraries like OpenCV, NumPy, and PyTorch provide essential tools for handling image data and implementing deep learning algorithms. The YOLO (You Only Look Once) architecture is used as the core object detection model because of its high speed and accuracy in detecting objects in real-time scenarios.

The system is designed to run on common operating systems such as **Windows, Linux, and macOS**, ensuring flexibility and portability across different development environments. These operating systems provide stable support for Python-based applications and machine learning frameworks. To facilitate efficient development and debugging, Integrated Development Environments (IDEs) such as **Visual Studio Code, PyCharm, and Jupyter Notebook** are used. These tools provide features such as code highlighting, debugging, and integrated terminals, making the development process faster and more organized. In addition to the development environment, the system relies on powerful machine learning frameworks such as **PyTorch**, which provides the backend support for training and executing the YOLO model. This framework allows efficient utilization of CPU and GPU resources for faster processing and real-time inference.

YOLO (You Only Look Once):

YOLO (You Only Look Once) is a state-of-the-art deep learning algorithm designed for real-time object detection. Unlike traditional detection methods that analyze images in multiple stages, YOLO performs object detection in a single pass through the neural network. This makes it significantly faster while maintaining high detection accuracy. The algorithm divides an image into grids and predicts bounding boxes and class probabilities for objects within those grids.

In this project, the YOLO model is used to detect underwater waste objects such as plastic bottles, gloves, fishing nets, metal waste, and other debris. The optimized YOLO model is trained using underwater datasets that contain labeled images of different types of marine waste. During the detection process, the model analyzes each frame from an image, video, or webcam feed and identifies objects by drawing bounding boxes and assigning confidence scores. Due to its speed and efficiency, YOLO is highly suitable for real-time applications such as underwater monitoring and environmental protection systems.

OpenCV:

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and image processing library widely used for real-time applications. It provides a comprehensive set of tools for image manipulation, object detection, video processing, and camera interfacing. OpenCV supports multiple programming languages, but it is most commonly used with Python for machine learning and artificial intelligence applications.

In this project, OpenCV plays an essential role in handling image and video inputs. It is used to capture frames from webcams or video files, preprocess underwater images, and display the detection results generated by the YOLO model. The preprocessing stage may include operations such as resizing images, enhancing contrast, reducing noise, and

correcting color distortions caused by underwater environments. OpenCV also helps visualize the output by drawing bounding boxes around detected objects and displaying the object labels along with confidence scores. This makes the detection results easy to interpret for users.

PyTorch:

PyTorch is an open-source deep learning framework developed by Facebook's AI Research lab. It provides a flexible and efficient platform for building, training, and deploying machine learning models. PyTorch is widely used in research and industry because of its dynamic computation graph, ease of debugging, and strong GPU acceleration capabilities.

In this project, PyTorch serves as the backend framework for implementing and running the YOLO object detection model. It enables the model to process large image datasets and perform complex computations required for training and inference. PyTorch also allows seamless integration with other Python libraries such as NumPy and OpenCV, making it ideal for computer vision applications. With GPU support, PyTorch significantly improves the speed of model training and real-time detection, ensuring that the system performs efficiently when processing live video streams or high-resolution images.

CHAPTER 3

SYSTEM ANALYSIS

CHAPTER 3

SYSTEM ANALYSIS

3.1 FEASIBILITY STUDY:

A feasibility study is conducted to evaluate the practicality and viability of the proposed Smart Underwater Waste Detection System based on the YOLO deep learning algorithm. The study analyzes the project from technical, economic, operational, and time feasibility perspectives to determine whether the system can be successfully developed, deployed, and maintained within the available resources and constraints. This evaluation ensures that the proposed marine waste detection system is realistic, sustainable, scalable, and aligned with current technological capabilities.

1. Technical Feasibility:

Technical feasibility evaluates whether the proposed underwater waste detection system can be implemented using the available technologies, software tools, and hardware infrastructure.

The core technology of the system is the YOLO (You Only Look Once) object detection algorithm, which is widely used for real-time object detection tasks. YOLO is built using Python and PyTorch, both of which are well-established and supported technologies. The availability of pretrained models, extensive documentation, and strong community support significantly simplifies implementation.

The system integrates YOLO with OpenCV for image preprocessing, video frame extraction, and annotation rendering. Preprocessing techniques such as contrast enhancement, resizing, normalization, and noise reduction improve detection performance in underwater conditions where visibility is low.

The backend is developed using Python and Flask, a lightweight web framework that enables seamless integration between the object detection model and the web interface. The frontend is designed using HTML, CSS, and JavaScript, ensuring compatibility with modern web browsers.

The system uses MySQL as the backend database for storing user credentials, uploaded files, detection results, timestamps, and statistical summaries. MySQL is a reliable and scalable relational database management system suitable for structured data storage. From a hardware perspective, the system can operate on standard computers with moderate specifications (8GB RAM or higher). While GPU acceleration improves real-time performance, the system can function efficiently on CPU using optimized YOLO variants. Overall, the availability of mature AI frameworks, web technologies, and database systems confirms that the proposed marine waste detection system is technically feasible.

2. Economic Feasibility

Economic feasibility determines whether the project can be developed and maintained within reasonable financial constraints while delivering significant value. The proposed system is highly cost-effective because it relies on open-source technologies. Python, Flask, YOLO, OpenCV, PyTorch, and MySQL are freely available and do not require licensing fees. This significantly reduces development costs compared to proprietary computer vision solutions.

The hardware requirements are moderate. The system can run on existing laboratory or personal computers without requiring expensive infrastructure. GPU usage is optional, making the system adaptable to different budget levels. Since pretrained YOLO models are used, the need for large-scale dataset collection and intensive model training is minimized. This reduces computational expenses and development time.

Maintenance costs are low due to the modular architecture of the application. Updates and improvements can be implemented without rebuilding the entire system. The system provides high value by enabling automated marine pollution monitoring, reducing manual inspection costs, and supporting environmental research initiatives. Therefore, the proposed system is economically feasible and sustainable for academic institutions, research organizations, and environmental agencies.

3. Operational Feasibility:

Operational feasibility examines whether the system can function effectively in real-world environments and be easily operated by users. The application provides a user-friendly web interface where users can register, log in, upload images or videos, and perform real-time detection. The interface is intuitive and does not require advanced technical knowledge.

The system supports three operational modes:

- ❖ Image detection
- ❖ Video detection with statistical analysis
- ❖ Real-time webcam detection

Detection results are displayed clearly with bounding boxes, object labels, confidence scores, and object counts. This ensures easy interpretation for researchers and environmental analysts.

The MySQL database stores detection history and user session data, allowing users to retrieve and analyze past records. This improves monitoring and environmental trend analysis. Although the system does not currently include automated waste removal mechanisms, it effectively fulfills its monitoring and detection objectives.

4. Time Feasibility:

Time feasibility evaluates whether the project can be completed within the allocated academic development period. The system is time-feasible because it uses pretrained YOLO models, eliminating the need for long training cycles and dataset creation from scratch. Development primarily involves integration of the detection model with the web framework and database.

The modular development approach allows parallel implementation of authentication, detection engine integration, media processing, and database configuration. This reduces overall development time. Extensive documentation and community support further simplify debugging and testing processes. Therefore, the project can be successfully completed within a standard academic timeline.

Feasibility Conclusion:

The feasibility study confirms that the Smart Underwater Waste Detection System is viable from technical, economic, operational, and time perspectives. The availability of reliable AI frameworks, open-source tools, structured database management, and scalable architecture ensures smooth development and deployment.

The project is cost-effective, easy to operate, and adaptable to real-world marine environments. It can be implemented within available resources while delivering accurate and real-time marine waste detection capabilities.

Overall, the proposed system is practical, sustainable, and well-suited for academic research and environmental monitoring applications, providing a strong foundation for future enhancements such as IoT integration and underwater drone deployment.

3.2 EXISTING SYSTEM:

Marine pollution monitoring in the existing system primarily relies on traditional and manual methods for identifying and managing underwater waste. These conventional approaches include physical inspections by divers, manual waste collection operations, boat-based surveys, satellite monitoring, and visual observation techniques. While these methods have contributed significantly to environmental conservation efforts, they suffer from several limitations in terms of efficiency, scalability, accuracy, and cost.

One of the most common existing approaches involves manual underwater inspection by divers. In this method, trained divers explore specific areas of oceans, rivers, or lakes to identify and document marine waste. Although this technique provides direct observation, it is labor-intensive, time-consuming, and potentially hazardous. Divers are exposed to risks such as strong currents, poor visibility, deep-water pressure, and harmful marine organisms. Additionally, manual inspection is limited to small geographic areas and cannot efficiently monitor large water bodies.

Another existing method includes the use of remotely operated vehicles (ROVs) equipped with cameras. These systems allow researchers to capture underwater images and videos. However, the detection and analysis of waste objects in captured footage are usually performed manually. Human operators must review large volumes of video data to identify pollutants, making the process slow and inefficient. Manual analysis is also prone to human error and inconsistency.

Satellite-based monitoring systems are also used to detect floating debris on the ocean surface. While satellites are effective for large-scale observation, they are limited in detecting underwater pollutants or smaller waste objects. Satellite imaging is also affected by weather conditions, cloud cover, and resolution constraints, reducing detection accuracy.

In many cases, pollution monitoring depends on community reporting and periodic surveys conducted by environmental organizations. These surveys provide valuable data but lack real-time detection capability. Data collection intervals may be long, resulting in delayed responses to pollution problems.

The existing systems generally lack automation and advanced data analytics. Most monitoring methods do not use artificial intelligence or machine learning for object detection. As a result, identifying specific waste categories requires manual classification, which reduces efficiency. Furthermore, there is often no centralized database for storing detection results, making long-term trend analysis difficult.

Another major limitation of existing systems is high operational cost. Hiring trained divers, operating boats or ROVs, and conducting large-scale surveys require significant financial resources. Smaller organizations and educational institutions may find it difficult to implement such systems.

Additionally, existing systems often lack user-friendly digital interfaces for researchers to access and analyze data easily. Data may be stored in reports or spreadsheets rather than structured databases, limiting scalability and data-driven decision-making.

In summary, the existing system for marine waste monitoring relies heavily on manual inspection, limited automation, and periodic surveys. These methods are labor-intensive, costly, time-consuming, and restricted in coverage. The absence of real-time detection, advanced analytics, and centralized data management highlights the need for an intelligent, automated solution to improve marine pollution monitoring efficiency and accuracy

3.3 DRAWBACKS OF THE EXISTING SYSTEM:

1. Heavy Dependence on Manual Effort

The system relies mainly on divers and human observers for detecting marine waste, which reduces efficiency.

2. Slow Monitoring Process

Manual surveys and video reviews take a long time, delaying pollution identification.

3. Limited Monitoring Range

Physical inspections can only cover small areas and cannot efficiently monitor vast ocean regions.

4. High Operational Expenses

Requires boats, diving equipment, trained personnel, and underwater vehicles, increasing overall cost.

5. No Real-Time Detection Capability

Existing systems do not provide instant alerts or live monitoring features.

6. Prone to Human Errors

Manual classification may result in incorrect identification or missed detection of pollutants.

7. Unsafe for Divers

Underwater inspections expose divers to risks such as strong currents, deep pressure, and marine hazards.

8. Inefficient Data Management

Lack of structured databases makes it difficult to store and analyze long-term pollution data.

9. Poor Detection in Low Visibility Conditions

Turbid water and poor lighting reduce observation accuracy.

10. Weather Dependency

Surface-level and satellite monitoring are affected by weather conditions and cloud cover.

11. Limited Detection of Submerged Waste

Satellite systems cannot detect waste that is fully submerged underwater.

12. No Automated Classification System

Waste categorization is done manually without artificial intelligence support.

13. Delayed Response to Pollution

Since surveys are periodic, pollution may spread before detection.

14. Difficult to Scale

Expanding monitoring operations requires additional manpower and equipment.

15. Inconsistent Reporting Standards

Data collected from different sources may lack uniformity and accuracy.

3.4 PROPOSED SYSTEM:

The proposed system introduces a Smart Underwater Waste Detection System designed to automatically identify and analyze marine pollutants using advanced deep learning techniques. The system leverages the YOLO (You Only Look Once) object detection algorithm to detect various types of underwater waste such as plastic bottles, fishing nets, face masks, and gloves from images, recorded videos, and real-time camera feeds. The primary goal of the proposed system is to provide an intelligent, automated, and efficient solution for monitoring marine pollution.

Unlike traditional monitoring methods that rely heavily on manual inspection and physical surveys, the proposed system utilizes artificial intelligence to detect waste objects quickly and accurately. YOLO is selected as the core detection algorithm due to its real-time processing capability and high detection accuracy. It processes the entire image in a single pass and simultaneously predicts bounding boxes, object classes, and confidence scores. This makes the system suitable for real-time underwater monitoring applications.

The proposed system is developed using the Flask web framework, which serves as the backend server for processing user requests. The application provides a web-based interface where users can securely log in and access detection features. The system includes a user authentication module with login and signup functionalities to ensure secure and controlled access. Passwords are stored using secure hashing mechanisms to protect user data.

The system supports three primary operational modes:

- Image Detection Mode – Users can upload underwater images, and the system processes them to identify and highlight waste objects using bounding boxes and confidence scores.
- Video Detection Mode – Users can upload video files, which are processed frame-by-frame. The system performs object detection on each frame and generates statistical summaries such as object count and detection frequency.
- Real-Time Detection Mode – The system utilizes webcam input to perform live detection, enabling instant identification of marine pollutants. This feature can be extended for integration with underwater cameras and robotic systems.

To enhance detection performance in challenging underwater environments, preprocessing techniques such as contrast enhancement, noise reduction, image resizing, and normalization are applied. These techniques improve image clarity and increase model accuracy in conditions affected by low light and water turbidity.

A MySQL database is integrated into the system to store user details, uploaded files, detection results, timestamps, and session statistics. This structured storage allows efficient data retrieval and long-term environmental analysis. Researchers and organizations can analyze stored records to study pollution trends and waste accumulation patterns. The proposed system is modular and scalable. It allows future improvements such as adding new waste categories, upgrading the YOLO model version, deploying the system on cloud

platforms, and integrating it with IoT devices and underwater drones for large-scale marine monitoring.

3.5 BENEFITS OF THE PROPOSED SYSTEM:

- **Real-Time Waste Detection**

The system is capable of detecting underwater waste objects in real time using the YOLO object detection algorithm. This allows immediate identification of marine pollutants from live camera feeds, enabling faster response and cleanup operations.

- **High Detection Accuracy**

By using advanced deep learning techniques, the system achieves high detection accuracy even in complex underwater environments. The YOLO model efficiently identifies objects such as plastic bottles, fishing nets, gloves, and masks with precise bounding boxes and confidence scores.

- **Automation of Monitoring Process**

Traditional marine pollution monitoring requires manual inspection by divers or researchers, which can be time-consuming and expensive. The proposed system automates the detection process, reducing human effort and improving operational efficiency.

- **Multiple Input Support**

Traditional marine pollution monitoring requires manual inspection by divers or researchers, which can be time-consuming and expensive. The proposed system automates the detection process, reducing human effort and improving operational efficiency.

- **Multiple Input Support**

The system supports different input sources including images, recorded videos, and real-time webcam feeds. This flexibility allows users to analyze underwater waste in various scenarios and environments.

- **Improved Image Quality through Preprocessing**

Preprocessing techniques such as contrast enhancement, noise reduction, and image normalization improve the quality of underwater images. These enhancements help the model detect objects more accurately in conditions with low visibility or poor lighting.

- **Secure User Access**

The system includes a user authentication module with login and signup functionality. This ensures that only authorized users can access the system and manage detection results, improving data security and system reliability.

- **Efficient Data Storage and Analysis**

The integration of a MySQL database allows the system to store user information, uploaded media files, detection results, timestamps, and statistical summaries. This stored data can be used for long-term analysis of pollution trends and environmental monitoring.

- **Scalable and Extendable System**

The proposed system is modular and scalable. New waste categories can be added, and the YOLO model can be upgraded to newer versions for improved accuracy and performance. The system can also be deployed on cloud platforms for large-scale environmental monitoring.

3.6 SCOPE OF THE PROJECT:

The scope of the proposed Smart Underwater Waste Detection System focuses on developing an intelligent solution for identifying and analyzing marine waste using deep learning and computer vision techniques. The system is designed to detect common underwater pollutants such as plastic bottles, fishing nets, gloves, face masks, and other debris from images, videos, and real-time camera feeds. By using the YOLO object detection algorithm, the system can automatically recognize and highlight these objects with bounding boxes and confidence scores, enabling efficient monitoring of marine environments.

The system is primarily intended for environmental monitoring and research purposes. It provides an automated approach for detecting underwater waste, reducing the need for manual inspection and physical surveys by divers or marine researchers. The integration of preprocessing techniques helps improve image clarity and detection accuracy in challenging underwater conditions such as low lighting, water turbidity, and color distortion.

The project also includes a web-based interface that allows users to upload images or videos and perform real-time detection using a webcam. The detection results are displayed visually and stored in a database for future analysis. This allows researchers and environmental organizations to track pollution patterns, monitor waste accumulation, and evaluate the effectiveness of cleanup operations.

In the future, the scope of this system can be extended by integrating it with underwater drones, robotic vehicles, or IoT-based monitoring devices for large-scale ocean surveillance. The system can also be enhanced by adding support for more waste categories, improving the deep learning model, and deploying the application on cloud platforms for remote access and large-scale environmental monitoring. Through these advancements, the proposed system can play an important role in marine pollution management and environmental conservation.

CHAPTER 4

SYSTEM DESIGN

CHAPTER 4

SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM:

The data flow diagram illustrates how data moves through the Smart Underwater Waste Detection System.

- User provides input (image/video/webcam) via frontend.
- Input data is sent to Flask Backend Server.
- Data flows into the Preprocessing Module.
- Processed data is passed to YOLO Detection Module.
- Detection results flow into Post-Processing Module.
- Refined results are stored in MySQL Database.
- Annotated output is displayed to the user.

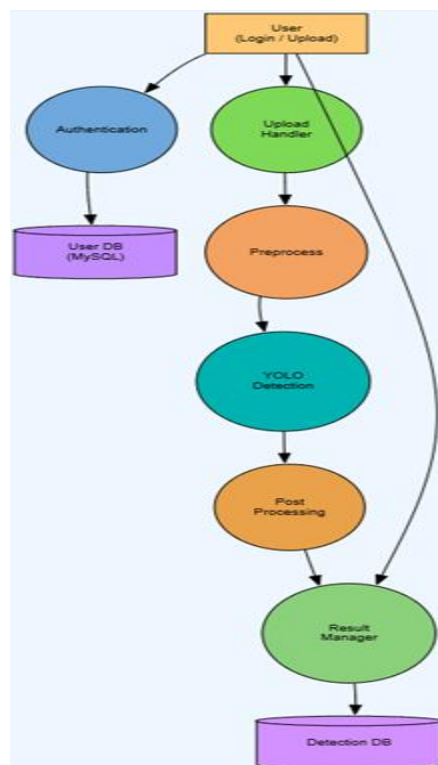


Fig 4.1 Data Flow Diagram

4.2 ARCHITECTURE DIAGRAM:

The system architecture diagram illustrates the complete workflow of the proposed Smart Underwater Waste Detection System based on the YOLO deep learning model. The process begins when a user accesses the web application through a browser interface developed using HTML, CSS, and JavaScript. The user logs into the system and uploads an underwater image, video file, or activates real-time webcam detection. The uploaded media is transmitted to the Flask backend server, where input validation and file handling operations are performed. The backend ensures secure authentication by verifying user credentials stored in the MySQL database. Once validated, the input data is forwarded to the preprocessing module.

In the preprocessing stage, operations such as image resizing, normalization, contrast enhancement, noise reduction, and frame extraction (for video or real-time streams) are performed using OpenCV. These preprocessing techniques improve image clarity and prepare the data for accurate detection under challenging underwater conditions.

The processed input is then passed to the YOLO object detection model. The model performs object classification and localization by generating bounding boxes, class labels (plastic bottle, fishing net, face mask, glove), and confidence scores. Post-processing techniques such as Non-Maximum Suppression (NMS) are applied to eliminate overlapping detections and refine the results.

Finally, annotated output images or processed video streams are displayed to the user through the frontend interface. Detection results, object counts, timestamps, and file paths are stored in the MySQL database for future analysis and environmental monitoring.

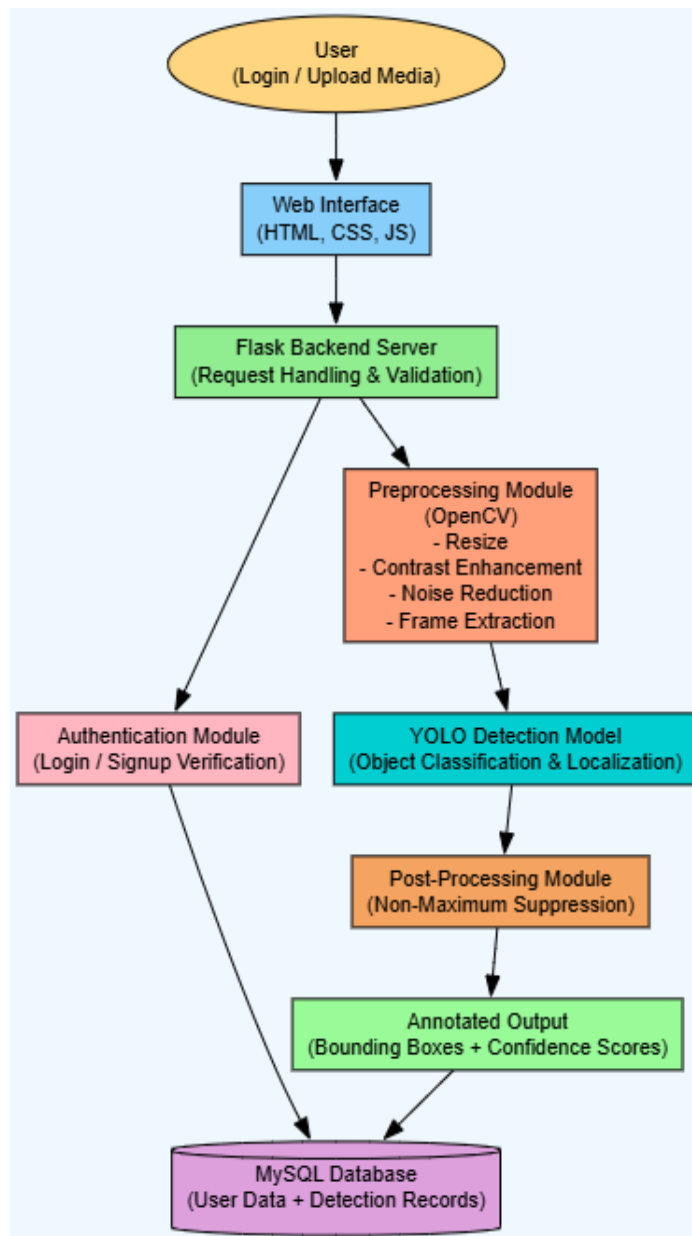


Fig 4.2 Architecture Diagram

4.2 DATABASE DESIGN:

The database plays an important role in the Smart Underwater Waste Detection System by storing user information, uploaded files, detection results, and system activity logs. A structured database is required to efficiently manage the data generated during the detection process and to allow easy retrieval of historical records for analysis.

In this project, a MySQL database is used to store and manage data. MySQL provides reliable performance, scalability, and efficient querying capabilities, making it suitable for handling structured data generated by the system. The database stores user authentication details, uploaded images and videos, detected object information, and timestamps for each detection event. The database is designed with multiple tables that are connected through relationships to maintain data integrity and avoid redundancy. The main tables used in the system are Users, Uploaded Files, Detection Results, and Detection Summary.

1. USERS TABLE:

This table stores the login and authentication details of users who access the system.

Field Name	Data Type	Description
user_id (PK)	INT	Unique ID for each user
username	VARCHAR(50)	Username of the user
email	VARCHAR(100)	User email address
password_hash	VARCHAR(255)	Encrypted password
created_at	DATETIME	Account creation time

2. UPLOADED FILES TABLE:

This table stores information about the images or videos uploaded by the users for waste detection

Field Name	Data Type	Description
file_id (PK)	INT	Unique file identifier
user_id (FK)	INT	Reference to Users table
file_name	VARCHAR(200)	Name of uploaded file
file_type	VARCHAR(20)	Image / Video
upload_time	DATETIME	Time of upload
file_path	VARCHAR(255)	Storage path of the file

3. DETECTION RESULTS TABLE:

This table stores the details of detected objects from the uploaded images or videos.

Field Name	Data Type	Description	
detection_id (PK)	INT	Unique detection ID	
file_id (FK)	INT	Reference to Uploaded_Files	
object_name	VARCHAR(50)	Detected object type (bottle, glove, net, etc.)	
confidence_score	FLOAT	Detection confidence value	
bounding_box	VARCHAR(100)	Coordinates of bounding box	
detected_time	DATETIME	Detection timestamp	

4. DETECTION SUMMARY TABLE:

This table stores summarized detection results for statistical analysis.

Field Name	Data Type	Description
summary_id (PK)	INT	Unique summary ID
file_id (FK)	INT	Reference to Uploaded_Files
total_objects	INT	Total detected objects
bottle_count	INT	Number of bottles detected
glove_count	INT	Number of gloves detected
net_count	INT	Number of nets detected
mask_count	INT	Number of masks detected
detection_date	DATE	Date of detection

CHAPTER 5

PROJECT DESCRIPTION

CHAPTER 5

PROJECT DESCRIPTION

5.1 OBJECTIVE:

The primary objective of the Smart Underwater Waste Detection System is to automatically identify and analyze marine pollutants using advanced deep learning and computer vision techniques. The system is designed to detect different types of underwater waste such as plastic bottles, fishing nets, gloves, face masks, and other debris from images, recorded videos, and real-time camera feeds. By utilizing the YOLO (You Only Look Once) object detection algorithm, the system aims to provide fast, accurate, and real-time detection of underwater waste objects.

Another key objective of the project is to reduce the dependency on manual inspection methods used in marine pollution monitoring. Traditional methods often require divers or researchers to physically survey underwater environments, which can be time-consuming, expensive, and sometimes risky. The proposed system automates the detection process, enabling faster identification of marine waste and supporting efficient environmental monitoring.

The system also aims to improve detection accuracy in challenging underwater conditions by applying image preprocessing techniques such as contrast enhancement, noise reduction, and image normalization. These techniques help the model perform better in environments with low visibility, poor lighting, and water turbidity. In addition, the project seeks to provide a user-friendly web-based interface that allows users to upload images, analyze videos, and perform real-time detection through a camera feed. The system stores detection results in a database, allowing researchers and environmental organizations to track pollution patterns and analyze waste accumulation trends over time.

5.2 MODULE DESCRIPTION:

1. User Authentication Module

The User Authentication Module ensures secure access to the Smart Underwater Waste Detection System. Since the application operates as a full-stack web platform handling user data and detection records, maintaining data privacy and restricting unauthorized access are critical.

During registration, users provide essential details such as username, email, and password. The system validates inputs to ensure completeness and correct email format. Duplicate accounts are prevented by verifying whether the username or email already exists in the MySQL database. Passwords are encrypted using secure hashing algorithms before storage. This prevents sensitive credentials from being stored in plain text, significantly improving security.

During login, the system compares the entered password (after hashing) with the stored encrypted password. If authentication is successful, a secure session is created using Flask session management. This session allows access to protected features such as:

- Image detection
- Video detection
- Real-time webcam detection
- Detection history access

Unauthorized users attempting to access protected routes are redirected to the login page. Logout functionality securely terminates sessions, preventing session hijacking or misuse. Additionally, input validation mechanisms help prevent security threats such as SQL injection and malicious form submissions. Overall, this module ensures secure user management, data protection, and controlled system access.

2. Frontend Interface Module

The Frontend Interface Module acts as the user interaction layer of the Smart Underwater Waste Detection System. It is developed using HTML, CSS, and JavaScript to provide a responsive and user-friendly experience.

The interface includes:

- Registration Page
- Login Page
- Dashboard
- Image Upload Page
- Video Upload Page
- Real-Time Detection Page
- Detection History Page

The layout is clean and intuitive, enabling researchers, students, and environmental organizations to use the system without technical complexity. JavaScript performs client-side validation for file format and file size before submission. Loading animations and status messages improve user interaction.

Detection results are displayed as:

- Annotated images/videos
- Bounding boxes around detected marine waste
- Object labels (plastic bottle, fishing net, mask, glove)
- Confidence scores
- Object count statistics

Color-coded bounding boxes make object identification easier.

Responsive design ensures compatibility across desktops and laptops.

Overall, the frontend enhances accessibility, usability, and smooth interaction between users and the detection engine.

3. Media Upload & Processing Module

The Media Upload & Processing Module handles user-provided input and prepares it for detection.

The system supports three input types:

- Image Upload (JPG, JPEG, PNG)
- Video Upload (MP4, AVI)

Live Webcam Stream:

- When media is uploaded, it is securely transmitted to the Flask backend and stored temporarily.

For images:

- Format validation is performed.
- Resolution adjustments are applied if necessary.

For videos:

- OpenCV extracts frames sequentially.
- Frames are processed individually for detection.
- Efficient memory handling prevents overload.

For real-time detection:

- Webcam frames are captured continuously.
- Processing is optimized to maintain minimal latency.

Preprocessing operations include:

- Resizing
- Contrast enhancement
- Noise reduction
- Normalization

These steps are especially important for underwater environments where visibility is poor due to water turbidity and lighting variations. Security measures ensure malicious files are rejected. This module ensures accurate data preparation, performance optimization, and safe file handling.

4. YOLO-Based Marine Waste Detection Module

This is the core module of the system. The YOLO (You Only Look Once) object detection model is used to identify marine pollutants such as:

- Plastic bottles
- Fishing nets
- Face masks
- Gloves

YOLO follows a single-stage detection architecture, meaning classification and localization occur in one forward pass. This ensures real-time detection capability.

When preprocessed input is received, the YOLO model Extracts spatial features
Predicts bounding box coordinates Assigns object class labels Generates confidence scores
For video and real-time detection, objects are tracked across frames to maintain detection consistency.

Confidence thresholds are applied to reduce false positives. Non-Maximum Suppression (NMS) removes overlapping bounding boxes.

Optimized YOLO variants allow the system to run on CPU systems, while GPU acceleration enhances performance. This module provides high accuracy, speed, and scalability for marine waste monitoring.

5. Database Management Module

The Database Management Module uses MySQL to store structured system data.

It manages:

- User authentication data
- Detection records
- Media file paths
- Detection statistics , Timestamps

The database contains structured tables such as:

- User Table
- Detection Table
- Media Table

Each detection record is linked to a specific user using foreign keys.

Instead of storing large media files directly, only file paths are saved. This improves efficiency and reduces storage overhead. Secure query handling prevents SQL injection. Proper indexing ensures fast retrieval of records. This module ensures data integrity, organization, and secure storage.

6. History & Report Module

The History & Report Module allows users to review previously processed detection results.

After each detection, the system stores:

- Input type
- Detected object classes
- Object count
- Confidence scores
- Timestamps
- Annotated output file path

Users can access their detection history through the dashboard Features include:

- Viewing detection details
- Filtering by date
- Reviewing object statistics
- Downloading annotated outputs

5.3 IMPLEMENTATION:

The implementation of the Smart Underwater Waste Detection System is carried out in several stages to ensure accurate detection of underwater waste using deep learning techniques. The system integrates image processing, machine learning, and web application technologies to provide an efficient and user-friendly solution.

Step 1: Dataset Collection

The first step in the implementation process is collecting a dataset containing underwater images of marine waste objects such as plastic bottles, fishing nets, gloves, face masks, and other debris. These images are gathered from various sources and represent different underwater environments and lighting conditions.

Step 2: Data Annotation

After collecting the dataset, each image is annotated using labeling tools. Bounding boxes are drawn around waste objects and labeled according to their respective classes. This annotated dataset is essential for training the YOLO object detection model.

Step 3: Image Preprocessing

Before training the model, preprocessing techniques are applied to improve image quality. These techniques include image resizing, contrast enhancement, noise reduction, and normalization. Preprocessing helps improve detection accuracy in challenging underwater conditions such as low visibility and color distortion.

Step 4: Model Training

In this stage, the YOLO model is trained using the prepared dataset. The model learns to recognize different types of underwater waste by analyzing the features present in the labeled images. The training process is performed using the PyTorch deep learning framework, which allows efficient computation and faster model learning.

Step 5: Model Testing

Once the model training is completed, the trained model is tested on new images or videos to evaluate its detection accuracy. The model predicts the objects present in the image and generates bounding boxes along with confidence scores.

Step 6: System Development using Flask

A web application is developed using the Flask framework to provide a user-friendly interface. The web interface allows users to log in securely and access the system features for object detection.

Step 7: Image Detection Module

Users can upload underwater images through the web interface. The system processes the uploaded image using the trained YOLO model and displays the detected waste objects with bounding boxes and confidence scores.

Step 8: Video Detection Module

Users can upload video files to the system. The system processes the video frame by frame and performs object detection on each frame. The detected objects are displayed along with a summary of object counts.

Step 9: Real-Time Detection Module

The system supports real-time detection using a webcam or camera feed. Each video frame is captured and processed by the YOLO model to detect underwater waste objects instantly.

Step 10: Database Integration

A MySQL database is integrated into the system to store user information, uploaded files, detection results, timestamps, and summary statistics. This enables efficient data storage and retrieval for future analysis.

CHAPTER 6

SYSTEM TESTING

CHAPTER 6

SYSTEM TESTING

6.1 TESTING DEFINITION:

Testing is the process of evaluating a software system to ensure that it functions correctly and meets the specified requirements. It involves executing the system under controlled conditions to identify errors, bugs, or inconsistencies that may affect the performance or reliability of the application. The main purpose of software testing is to verify that all modules of the system operate as expected and to ensure that the final product is reliable, efficient, and free from major defects.

In the Smart Underwater Waste Detection System, testing is performed to ensure that the system accurately detects underwater waste objects from images, videos, and real-time camera feeds. The testing process also verifies the functionality of the user interface, authentication system, database operations, and object detection modules. Proper testing helps ensure that the system performs efficiently under different conditions and provides accurate detection results.

6.2 TESTING OBJECTIVE:

The main objective of testing in the Smart Underwater Waste Detection System is to ensure that the application works correctly and meets the functional requirements of the system. Testing is performed to verify that each module of the system performs its intended task without errors and produces reliable outputs.

The testing process aims to identify and fix any issues in the system before deployment. It ensures that the YOLO model correctly detects underwater waste objects and that the

detection results are displayed properly to the user. Another objective is to confirm that the system can handle different input formats such as images, videos, and real-time camera feeds without failure.

Testing also ensures that the database operations function correctly, including storing user information, uploaded files, and detection results. Additionally, the testing process verifies that the user authentication system works properly and that only authorized users can access the application. Overall, the objective of testing is to improve the quality, performance, reliability, and usability of the system.

6.3 TYTPES OF TESTING:

Several types of testing are performed to ensure that the Smart Underwater Waste Detection System functions properly and delivers accurate results.

1. Unit Testing

Unit testing is performed to test individual components or modules of the system separately. Each module, such as image preprocessing, object detection, database operations, and user authentication, is tested independently to ensure that it functions correctly.

2. Integrating Testing

Integration testing is carried out to verify that different modules of the system work together properly. In this project, integration testing ensures that the YOLO detection module works correctly with the Flask web application, database system, and image processing components.

3. Functional Testing

Functional testing verifies that the system performs according to the specified functional requirements. This includes testing image detection, video detection, real-time detection, user login, file uploads, and displaying detection results.

4. System Testing

System testing is performed on the complete application to evaluate its overall functionality and performance. It ensures that all modules are integrated correctly and that the system works smoothly as a whole.

5. Performance Testing

Performance testing evaluates the system's ability to process images and videos efficiently. It ensures that the YOLO model performs object detection quickly and that the system can handle multiple inputs without significant delays.

6. User Interface Testing

User interface testing ensures that the web application interface is easy to use and functions correctly. It verifies that users can upload files, view detection results, and navigate through the system without errors.

CHAPTER 7

CONCLUSION

CHAPTER 7

CONCLUSION

7.1 SUMMARY:

The Smart Underwater Waste Detection System is developed to automatically detect and analyze marine waste using advanced deep learning and computer vision techniques. Marine pollution caused by plastic bottles, fishing nets, gloves, face masks, and other debris has become a major environmental concern affecting aquatic ecosystems. Traditional monitoring methods often rely on manual inspection and underwater surveys, which can be time-consuming, costly, and inefficient. The proposed system provides an intelligent and automated solution to address this issue.

The system utilizes the YOLO (You Only Look Once) object detection algorithm to identify underwater waste objects from images, recorded videos, and real-time camera feeds. YOLO is chosen for this project because of its ability to perform fast and accurate object detection in real-time applications. The model processes the entire image in a single pass and predicts the location of objects along with their class labels and confidence scores. To improve detection accuracy in underwater environments, image preprocessing techniques such as contrast enhancement, noise reduction, and image normalization are applied. These techniques help reduce visual distortions caused by water conditions such as low lighting, turbidity, and color imbalance.

The system is implemented using Python along with libraries such as OpenCV, PyTorch, and the YOLO framework. A web-based interface developed using Flask allows users to upload images, analyze videos, and perform real-time detection through a webcam. The system also includes user authentication features to ensure secure access. Additionally, a MySQL database is used to store user information, uploaded files, detection results, and timestamps for future analysis.

Overall, the project demonstrates how artificial intelligence and deep learning technologies can be effectively used to monitor marine pollution and support environmental conservation efforts. The proposed system provides a scalable and efficient platform for detecting underwater waste and can be further enhanced for integration with underwater drones, robotic systems, and large-scale environmental monitoring applications.

7.2 FUTURE ENHANCEMENT:

Although the Smart Underwater Waste Detection System provides an efficient solution for detecting marine waste, several improvements can be made in the future to enhance its performance, functionality, and scalability. These enhancements will help expand the system's capabilities and make it more suitable for large-scale environmental monitoring. One of the major future enhancements is the integration of the system with underwater drones or robotic vehicles. These devices can capture real-time images and videos from deep-sea environments, allowing the detection system to monitor large ocean areas automatically without requiring human intervention.

Another improvement would be the expansion of the dataset and object categories. Currently, the system focuses on detecting common underwater waste such as plastic bottles, fishing nets, gloves, and face masks. In the future, the system can be trained to detect additional types of marine debris such as plastic bags, cans, glass bottles, and other pollutants to increase the accuracy and coverage of waste detection.

The system can also be enhanced by **upgrading the object detection model to newer and more advanced versions of YOLO or other deep learning architectures**. These advanced models may provide better detection accuracy, faster processing speed, and improved performance in challenging underwater conditions.

Another potential improvement is **cloud-based deployment**. By deploying the system on cloud platforms, users from different locations can access the application through the

internet. This will allow environmental organizations and researchers to monitor marine pollution remotely and store large volumes of detection data for analysis. Additionally, the system can be integrated with **Geographic Information Systems (GIS)** to map detected waste locations. This would help researchers track pollution hotspots and plan effective cleanup operations in affected marine areas. Overall, these future enhancements will make the system more powerful, scalable, and useful for real-world applications in marine pollution monitoring and environmental conservation.

CHAPTER 8

APPENDIX

CHAPTER 8

APPENDIX

8.1 SCREENSHOTS:

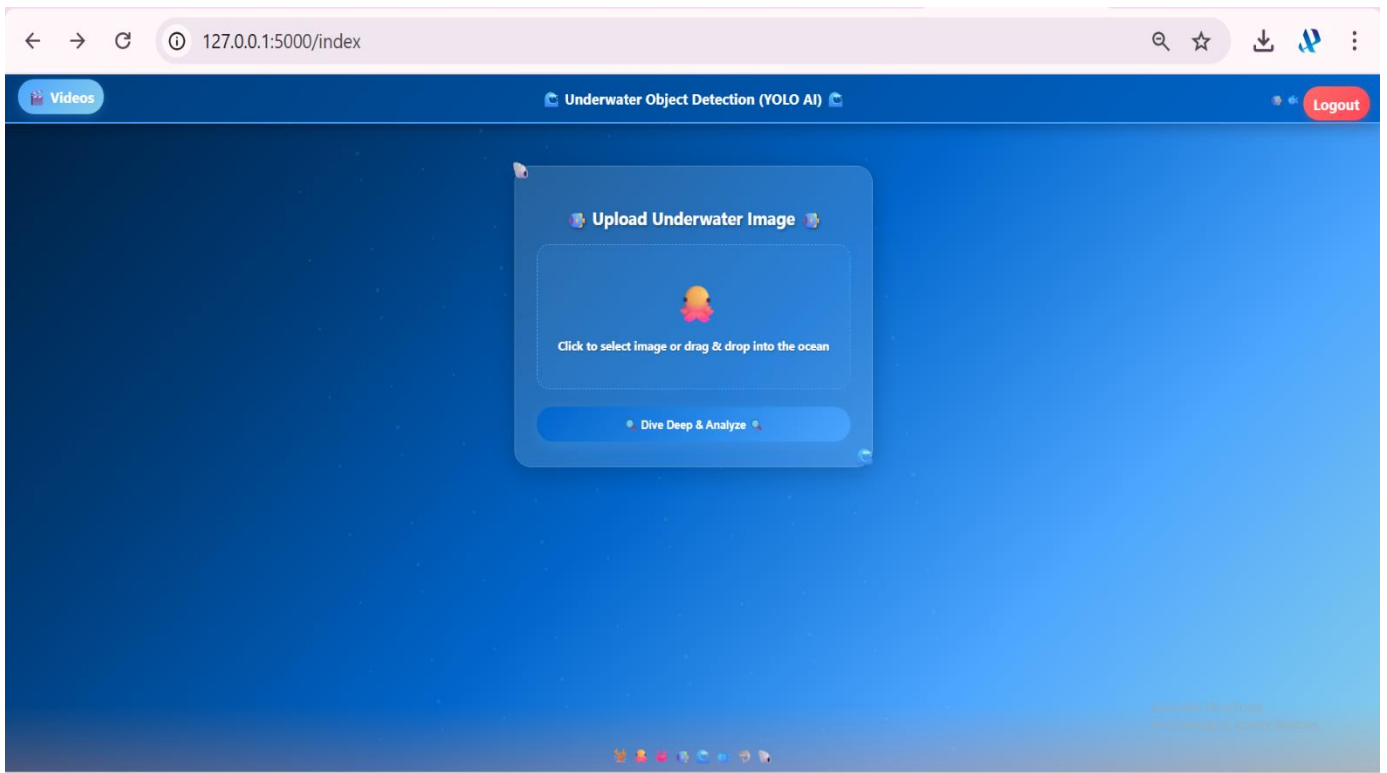


Fig 8.1 Flask HomePage Interface

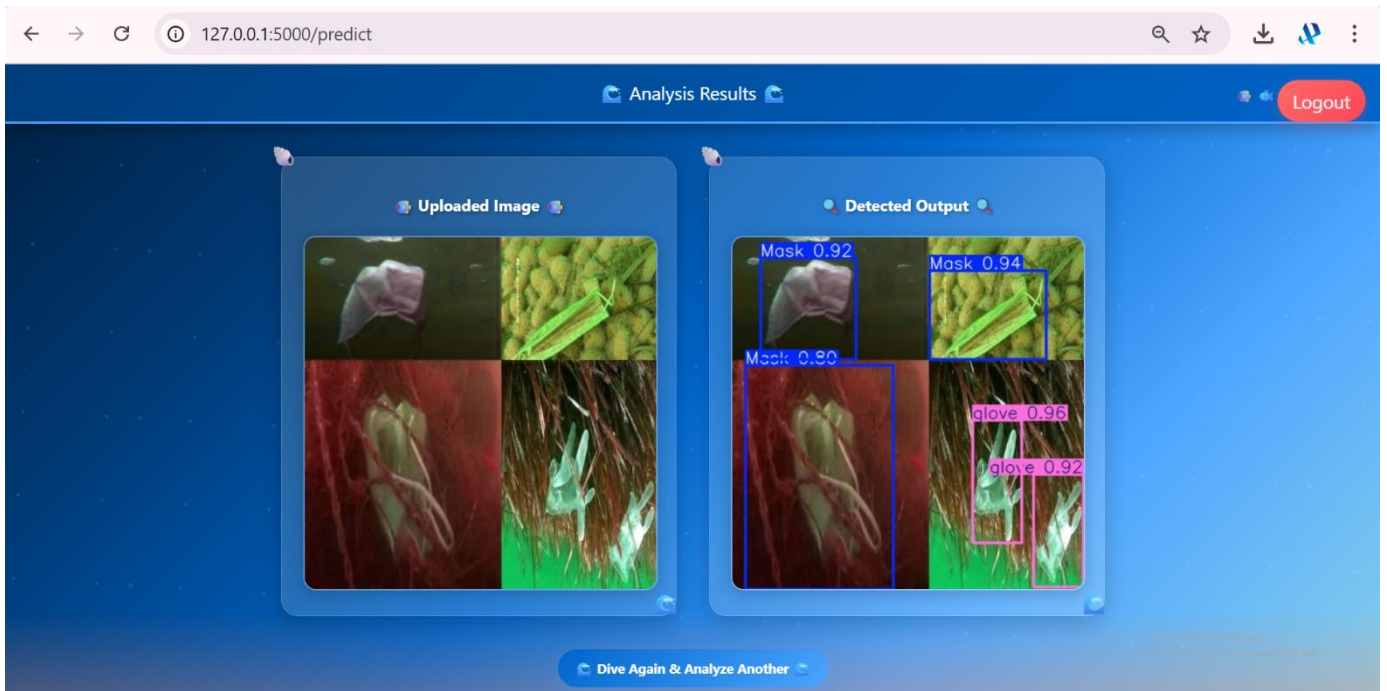


Fig 8.1 Interface After Uploading Image

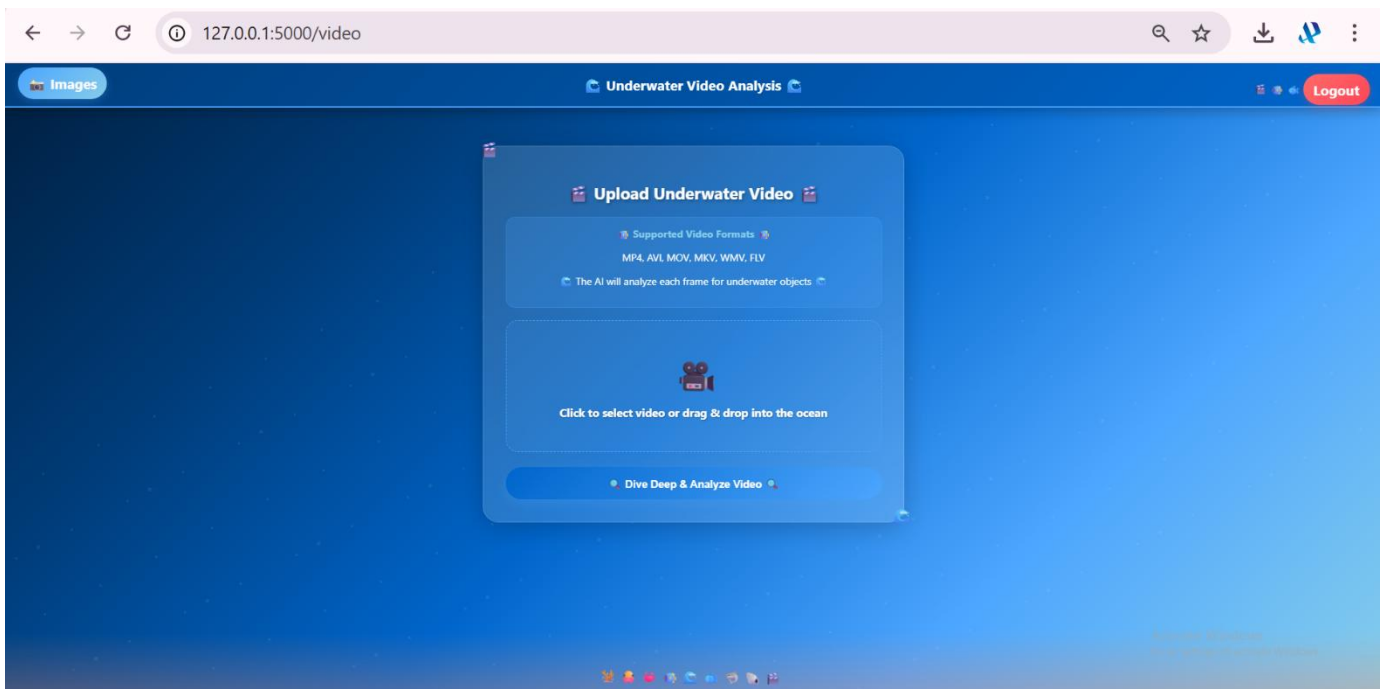


Fig 8.1 Interface Of Video Uploading

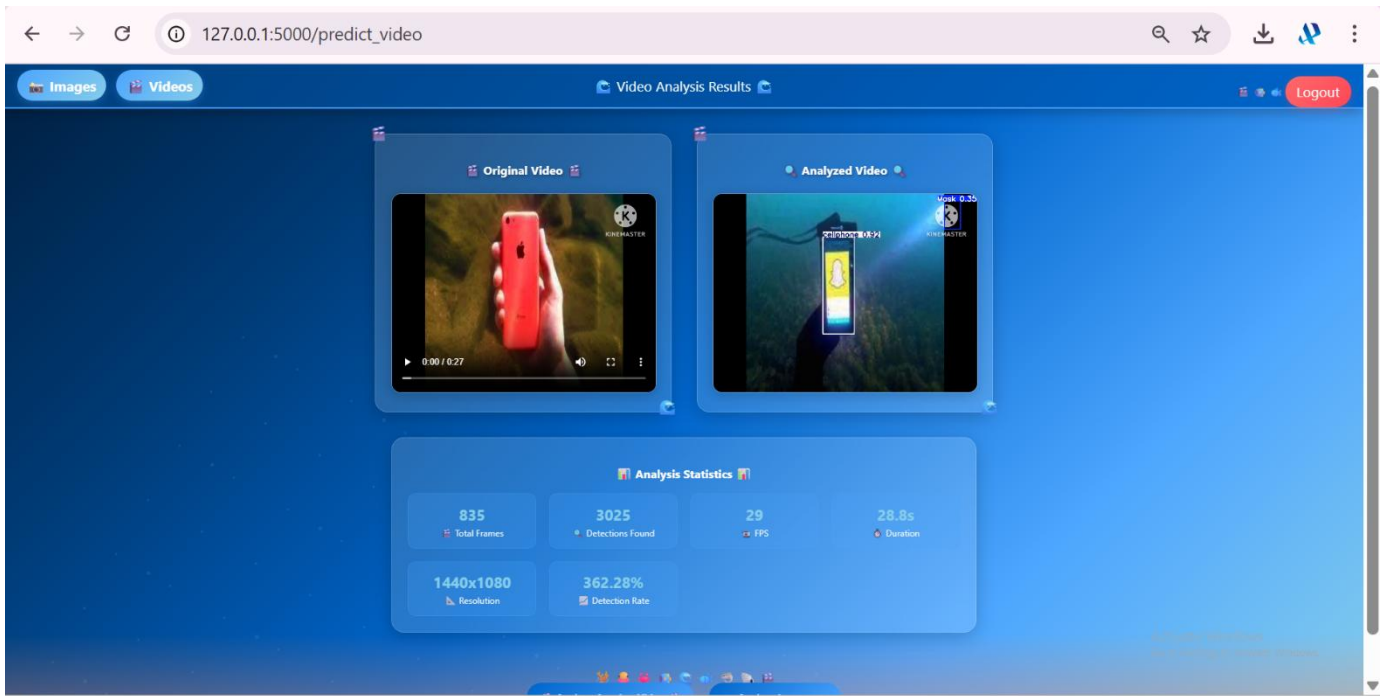


Fig 8.1 After Detecting The Video

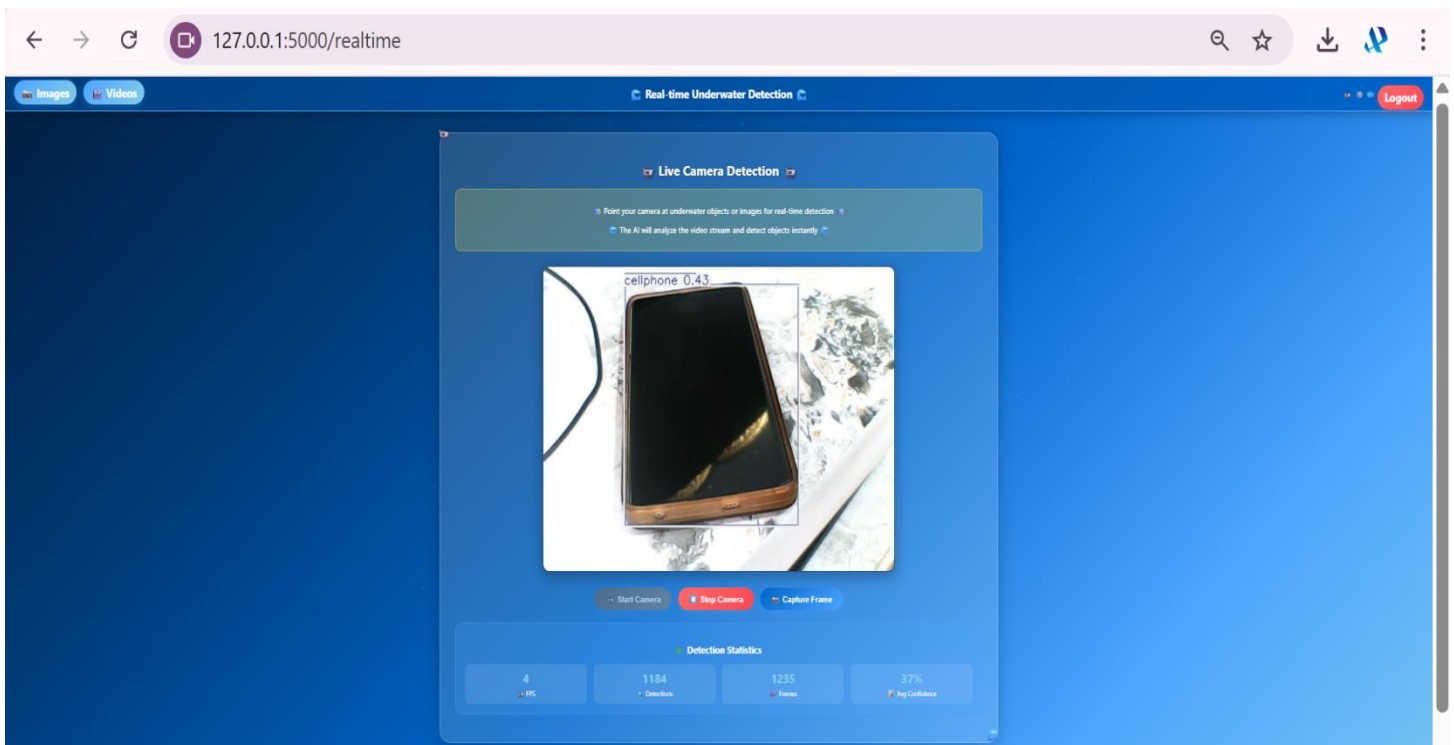


Fig 8.1 Interface Of Real-time Camera Detection

8.2 CODING:

```
from flask import Flask, render_template, request, redirect, session, url_for, jsonify
from ultralytics import YOLO
import os
import shutil
import cv2
import base64
from video_processor import DentalVideoProcessor, get_video_info
from werkzeug.utils import secure_filename

app = Flask(__name__)
app.secret_key = "underwater"

users = {}

# Load model
model = YOLO(r'C:\Users\dhaksheshh\Downloads\final underwater\final
underwater\best.pt')

# Initialize video processor
video_processor = DentalVideoProcessor(r'C:\Users\dhaksheshh\Downloads\final
underwater\final underwater\best.pt')

# Proper Flask paths
UPLOAD_FOLDER = 'static/uploads'
OUTPUT_FOLDER = 'static/output'
```

```

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(OUTPUT_FOLDER, exist_ok=True)

# Allowed file extensions
ALLOWED_IMAGE_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'bmp'}
ALLOWED_VIDEO_EXTENSIONS = {'mp4', 'avi', 'mov', 'mkv', 'wmv', 'flv'}

def allowed_file(filename, file_type='image'):
    if file_type == 'image':
        return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_IMAGE_EXTENSIONS
    elif file_type == 'video':
        return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_VIDEO_EXTENSIONS
    return False

@app.route('/')
def home():
    if 'user' in session:
        return redirect('/index')
    return redirect('/login')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

```

```

    if email in users and users[email] == password:
        session['user'] = email
        return redirect('/index')

    return render_template('login.html', error="Invalid email or password")

return render_template('login.html')

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        if email in users:
            return render_template('signup.html', error="Email already exists")

        users[email] = password
        return redirect('/login')

    return render_template('signup.html')

@app.route('/logout')
def logout():
    session.pop('user', None)
    return redirect('/login')

@app.route('/index')

```

```

def index():
    if 'user' not in session:
        return redirect('/login')
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    if 'user' not in session:
        return redirect('/login')

    file = request.files['image']
    filename = secure_filename(file.filename)

    if not allowed_file(filename, 'image'):
        return render_template('index.html', error="Please upload a valid image file")

    # Save upload into static/uploads
    upload_path = os.path.join(UPLOAD_FOLDER, filename)
    file.save(upload_path)

    # Run YOLO
    results = model(upload_path, conf=0.25, save=True)

    # YOLO output location
    yolo_output_path = results[0].save_dir
    detected_file_path = os.path.join(yolo_output_path, filename)

    # Copy YOLO result into static/output

```

```

final_output_path = os.path.join(OUTPUT_FOLDER, filename)
shutil.copy(detected_file_path, final_output_path)

# RETURN CLEAN PATHS FOR HTML
return render_template(
    'result.html',
    input_image=f'/static/uploads/{filename}',
    output_image=f'/static/output/{filename}'
)

@app.route('/video')
def video_upload():
    """Video upload page"""
    if 'user' not in session:
        return redirect('/login')
    return render_template('video.html')

@app.route('/predict_video', methods=['POST'])
def predict_video():
    """Handle video upload and processing"""
    if 'user' not in session:
        return redirect('/login')

    if 'video' not in request.files:
        return render_template('video.html', error="No video file selected")

    file = request.files['video']
    if file.filename == "":

```

```

return render_template('video.html', error="No video file selected")

filename = secure_filename(file.filename)

if not allowed_file(filename, 'video'):
    return render_template('video.html', error="Please upload a valid video file (mp4, avi,
mov, mkv, wmv, flv)")

# Save uploaded video
upload_path = os.path.join(UPLOAD_FOLDER, filename)
file.save(upload_path)

print(f"📁 Video uploaded: {upload_path}")
print(f"📏 File size: {os.path.getsize(upload_path) / (1024*1024):.2f} MB")

# Get video info
video_info = get_video_info(upload_path)
if not video_info:
    return render_template('video.html', error="Could not process video file")

print(f"📺 Video info: {video_info}")

# Process video
output_filename = f"processed_{filename}"
result = video_processor.process_video(upload_path, output_filename)

print(f"✅ Processing result: {result}")

```

```

if result['success']:
    # Verify output file exists and has content
    output_path = result['output_video']
    if os.path.exists(output_path):
        output_size = os.path.getsize(output_path) / (1024*1024)
        print(f"✅ Output video created: {output_path}")
        print(f"📄 Output size: {output_size:.2f} MB")
    else:
        print(f"❌ Output video not found: {output_path}")
        return render_template('video.html', error="Output video was not created")

return render_template(
    'video_result.html',
    input_video=f"/static/uploads/{filename}",
    output_video=f"/static/output/{output_filename}",
    stats=result['stats'],
    video_info=video_info
)
else:
    error_msg = f"Video processing failed: {result.get('error', 'Unknown error')}}"
    print(f"❌ {error_msg}")
    return render_template('video.html', error=error_msg)

@app.route('/video_status/<filename>')
def video_status(filename):
    """Check video processing status (for future async processing)"""

```

```

if 'user' not in session:
    return jsonify({'error': 'Not authenticated'}), 401

# This can be expanded for real-time processing status
output_path = os.path.join(OUTPUT_FOLDER, f'processed_{filename}')

if os.path.exists(output_path):
    return jsonify({'status': 'completed', 'output_path':
f'/static/output/processed_{filename}'})
else:
    return jsonify({'status': 'processing'})

@app.route('/realtime')
def realtime():
    """Real-time camera detection page"""
    if 'user' not in session:
        return redirect('/login')
    return render_template('realtime.html')

@app.route('/detect_frame', methods=['POST'])
def detect_frame():
    """Process a single frame from webcam for real-time detection"""
    if 'user' not in session:
        print("✗ detect_frame: User not authenticated")
        return jsonify({'error': 'Not authenticated'}), 401

    try:
        if 'frame' not in request.files:

```

```

print("❌ detect_frame: No frame in request")
return jsonify({'success': False, 'error': 'No frame provided'})

frame_file = request.files['frame']

print(f"📄 detect_frame: Received frame, size: {len(frame_file.read())} bytes")
frame_file.seek(0) # Reset file pointer after reading

# Save frame temporarily
temp_path = os.path.join(UPLOAD_FOLDER, 'temp_frame.jpg')
frame_file.save(temp_path)
print(f"💾 detect_frame: Saved to {temp_path}")

# Run YOLO detection
results = model(temp_path, conf=0.25, verbose=False)

# Get detection count and confidence
detections = len(results[0].boxes)
avg_confidence = 0

if detections > 0:
    confidences = [float(box.conf) for box in results[0].boxes]
    avg_confidence = sum(confidences) / len(confidences)
    print(f"🔍 detect_frame: Found {detections} detections, avg confidence: {avg_confidence:.2f}")
else:
    print(f"🔍 detect_frame: No detections found")

```

```

# Get annotated image as base64
annotated_frame = results[0].plot()
_, buffer = cv2.imencode('.jpg', annotated_frame)
img_base64 = base64.b64encode(buffer).decode('utf-8')

# Clean up temp file
if os.path.exists(temp_path):
    os.remove(temp_path)

return jsonify({
    'success': True,
    'detections': detections,
    'confidence': avg_confidence,
    'image': img_base64
})

except Exception as e:
    print(f"❌ Error in detect_frame: {str(e)}")
    import traceback
    traceback.print_exc()
    return jsonify({'success': False, 'error': str(e)})

if __name__ == '__main__':
    app.run(debug=True)

```

BIBLIOGRAPHY AND REFERENCES:

1. Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi, **“You Only Look Once: Unified, Real-Time Object Detection,”** Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
2. OpenCV Documentation. Available at: <https://opencv.org> Accessed for image processing and computer vision operations.
3. PyTorch Documentation. Available at: <https://pytorch.org> Used for implementing and training the YOLO deep learning model.
4. Flask Documentation. Available at: <https://flask.palletsprojects.com> Used for developing the web-based interface and backend server.
5. MySQL Documentation. Available at: <https://www.mysql.com> Used for storing user data, uploaded files, and detection results.
6. Python Documentation. Available at: <https://www.python.org> Used as the main programming language for system development.
7. YOLOv5 GitHub Repository by Ultralytics. Available at: <https://github.com/ultralytics/yolov5>
8. Research articles and online resources related to **marine pollution monitoring and underwater object detection** were also referred to during the development of this project.